Ocean Engineering 308 (2024) 118304



Contents lists available at ScienceDirect

Ocean Engineering

journal homepage: www.elsevier.com/locate/oceaneng





A bi-layer task decision-making approach for USV clusters with multi-constrained task points

Xiaoming Zhao, Rui Song *, Yangmin Xie, Zihao Wang, Yan Peng

School of Future Technology, Shanghai University, China
Institute of Artificial Intelligence, Shanghai University, China

ARTICLE INFO

Keywords:
Unmanned surface vehicle (USV)
Task decision-making
Task planning
Self-organizing map (SOM)
Multi-constrained task points

ABSTRACT

The Unmanned Surface Vehicles (USVs) cluster shows significant potential in executing complex maritime tasks. However, existing task decision-making approaches often overlook inherent constraints of tasks such as executable time and operational area, leading to inefficiency during execution. This paper addresses the task decision-making problem for USV clusters by a bi-layer framework and introduces a novel Multi-Constrained Task Point Adaptive Self-Organizing Map (MCTP-ASOM) algorithm to improve the task execution efficiency within these clusters. In the task planning layer, the MCTP-ASOM algorithm incorporates the dynamic time window and perceptual distance constraints of task points. It adapts neuron movement by incorporating task waiting time and perceptual distance into the SOM algorithm's WTA and Kohonen's rules, enabling the planning of task execution sequences and optimization of execution positions, effectively reducing average waiting time and navigation costs during task completion. In the multi-task allocation layer, the K-means algorithm is utilized to minimize the inter-task distance assigned to each USV, thereby optimizing the cluster efficiency. Extensive simulations across various scenarios confirm the effectiveness of this approach, with an average reduction in task waiting time by 18.34%, per USV waiting time by 18.86%, and navigation cost by 16.3%, particularly beneficial in multi-USV operations with various constraints.

1. Introduction

Given the increasing global emphasis on marine resource development and maritime territorial preservation, the advancement of intelligent unmanned maritime equipment has great international attention. Unmanned surface vehicles (USVs), distinguished by their compact size, lightweight design, high autonomy, and extensive operational capabilities, are now the preferred instruments for oceanic exploration and development worldwide (Liu et al., 2016; Ma et al., 2018). Unlike traditional vessels, USVs can automatically execute fundamental tasks. However, the complex marine environment and task diversity present challenges for individual USVs including limited operational ranges and task type restrictions. These limitations hinder their ability to fulfill practiced task requirements effectively.

To overcome these challenges, the current trend in USV research leans towards employing USV clusters operating in collaboration. A USV cluster, composing multiple USVs with basic autonomous functionalities, utilizes the distributed system functions across multiple units to achieve a scale effect. It offers benefits such as reduced task completion times, extended operational ranges, and enhanced resilience. The collaborative operation of multiple USVs can be applied in both military (maritime patrols, mine sweeping, and reconnaissance) and civilian (maritime waste cleanup, hydrological and meteorological survey, search and rescue) domains, demonstrating their wide-ranging potential range of potential applications (Zereik et al., 2018).

The presence of coastal islands, the intricate and dynamic marine environment, and other factors introduce uncertainty into maritime tasks, significantly enhancing the challenge for USV clusters to safely execute these tasks. To ensure the successful completion of maritime tasks and minimize resource consumption by USVs during execution, effective task decision-making becomes crucial in the context of multi-USV cooperation. Task decision-making for multiple USVs is not simply a matter of combining USVs, which involves a typical non-deterministic polynomial hard (NP-hard) problem. Multi-USV task decision-making involves assigning one or multiple tasks to each USV within the cluster, considering specific environmental information and task requirements, while also accounting for the limitations of each USV's capabilities, the aim is to avoid resource conflicts and wastage, thereby maximizing the efficiency of cluster task execution, in addition to providing feasible movement trajectories for multiple USVs based on the task allocation scheme. To expedite task decision-making for multiple USVs,

* Corresponding author at: School of Future Technology, Shanghai University, China.
E-mail address: song_rui@shu.edu.cn (R. Song).

https://doi.org/10.1016/j.oceaneng.2024.118304
Received 26 March 2024; Received in revised form 11 May 2024; Accepted 26 May 2024
Available online 3 June 2024
0029-8018/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

it is imperative to decompose the complex decision-making problem into simpler sub-problems through decoupling. Task decision-making inherently involves providing each USV with detailed allocation results and execution paths based on task requirements (Park et al., 2021). Consequently, prevalent task decision-making methodologies are systematically divided into two progressive components: multi-task allocation and task planning. These components are carefully designed to provide each USV with the detailed information it needs to complete tasks.

Task allocation is the basis for task decision-making in USV clusters, addressing the problem of how to reasonably distribute multiple USVs to task points (Xue et al., 2021). In this process, the type, range, location, and communication range of the USVs need to be taken into account to decide which tasks should be assigned to specific USVs. To acquire the set of tasks for each USV, Xia et al. (2021) employed the self-organizing map (SOM) algorithm. They consider each USV as a "cluster center" within the SOM and utilize the competitive learning mechanism to manage multi-task allocations within a designated task region. In MahmoudZadeh and Yazdani (2022), the agglomerative hierarchical clustering algorithm (AHCA) incorporating Ward's criteria of using distance measurements between buoys as a clustering criterion, combined with the environmental model and the number of USVs, this approach efficiently divides the operational area into k exclusive partitions, ensuring conflict-free operation, i.e., no task overlap. Xue et al. (2021) precise algorithm constructs a cost matrix for task allocation by considering the time and turning time of the initial and target points through the planned path, aiming to minimize the maximum task time and total time. Addressing task allocation in multi-robot systems, an improved SOM-based approach is proposed in Zhu and Yang (2010), focusing on the self-organizing problem of a large number of robots and task locations in a dynamic environment. To tackle energy consumption and communication range issues, Liu et al. (2019) have introduced a new energy coordination scheme and task prioritization method, effectively assigning tasks to multiple USV systems.

Task planning is another important module of the task decision-making technique, which requires planning the optimized task execution sequence and path. for each USV by taking into account the type of task, distance, priority, and other factors after achieving task allocation. In Jin et al. (2022), a membrane reward mechanism was combined with a genetic algorithm to achieve cooperative multi-task traversing planning considering task priorities. Liu and Bucknall (2018) modeled multi-target task planning as a traveling salesman problem (TSP) and proposed an intelligent algorithm for multi-task allocation and task planning. This algorithm integrates an adaptive artificial repulsive field into a SOM, enabling the rapid and effective generation of task execution sequence in complex maritime environments. An efficient decentralized planning algorithm named Dec-SOM is introduced for multi-robot information collection problems in Best and Hollinger (2020). Each robot plans its path points through the SOM algorithm, achieving decentralization by repeatedly communicating its plans. However, it is worth noting that this algorithm demands high communication requirements and may not be suitable for large-scale, complex, and highly turbulent sea conditions. Subsequently, since the route traveled by the USV must be feasible and safe, it is necessary to avoid obstacles such as islands and buoys in the ocean during the task planning process. Common USV obstacle avoidance planning methods include rapidly exploring random trees algorithm (Mao et al., 2023), A* algorithm (Singh et al., 2018), artificial potential field method (Wang et al., 2021), genetic algorithm (Kim et al., 2017), and so on. Recently, there has been a growing interest in combining collaborative avoidance planning problems with reinforcement. Woo and Kim (2020) modeled the USV collision avoidance problem as a Markov decision process and considering the convention on the collision regulations for the prevention of collisions at sea (COLREGS), a reward function is designed for the deep reinforcement learning network, trained a model capable of selecting appropriate evasion strategies based on encounter types.

While breaking down the intricate task decision-making problem into multiple sub-problems for resolution significantly reduces computational complexity, the diversity of maritime task requirements poses higher demands on the effective operation of USV clusters. In previous research, scholars have focused on marine environmental factors, such as in Ma et al. (2021), where obstacles' impact on USV communication distance was considered during decision-making. A signal propagation model was integrated into a weighted competition strategy to minimize communication hindrances caused by obstacles when allocating tasks. Considering the influence of ocean currents, a neural network-based task allocation algorithm was proposed in Zhu et al. (2019), effectively assigning tasks to a group of underwater robots to minimize energy consumption. Scholars also recognized the heterogeneous energy consumption constraints within the USV cluster. In Tan et al. (2022), a new fitness function was designed to quantify the heterogeneity of USVs in terms of task execution capability and energy reserves. Based on this fitness function, a novel candidate selection method was proposed for SOM to allocate multiple types of tasks to specific USVs. Considering task priority constraints, a new genetic optimization algorithm was introduced in Jin and Er (2022) to achieve priority task allocation. Under the precondition of prioritizing rescue targets, the algorithm ensures the safe return of all targets while optimizing path length, navigation time, and angular energy. Additionally, Jin et al. (2022) proposed a membrane reward mechanism genetic optimization algorithm based on genetic algorithm to expedite task allocation speed and implement prioritization, addressing unique task allocation challenges in complex ocean environments. In general, these studies underscore the importance of addressing marine environmental characteristics, optimizing path energy consumption, and the priority among task points. However, current multi-USV collaborative task decision-making technologies still have limitations in fully considering the impact of inherent task constraints on task execution costs. The temporal-sensitive nature of tasks poses challenges to the efficient execution of tasks by USV clusters, requiring completion within specific timeframes. Existing task planning methods often overlook temporal sensitivity, resulting in delayed responses during emergencies or causing a waste of temporal resources due to premature action. For example, in marine data collection missions, if USVs arrive before the data collection window opens, early-arriving USVs may need to wait in the task area, leading to unnecessary energy and time consumption that could have been allocated to other tasks. Additionally, the operational area of tasks presents another constraint, planning optimal execution paths based on task operational areas to minimize navigation costs is crucial in the decision-making process of USV clusters. Failure to adequately address these inherent constraints associated with tasks, such as the task's executable time and operational area constraints, may result in suboptimal decision outcomes, impacting overall task efficiency. In conclusion, to improve the task execution efficiency of USV clusters in complex maritime environments, there is a pressing need to develop a more advanced task decision-making approach.

This paper focuses on the problem of information-gathering on a given number of task points in a given closed sea area by utilizing USV clusters. A novel bi-layer task decision-making solution framework is proposed to mitigate the computational complexity associated with task decision-making. The K-means algorithm is employed for efficient task allocation, aimed at minimizing the inter-task distance assigned to each USV, thereby optimizing the overall cluster efficiency. As shown in Fig. 1, maritime surface task points in this paper have dual constraints: a time window constraint and a certain perceptual distance constraint. This paper innovatively introduces the Multi-Constrained Task Point Adaptive Self-Organizing Map (MCTP-ASOM) algorithm, which comprehensively considers the inherent constraints of task points. This MCTP-ASOM algorithm takes into account the dynamic time window constraints of task points, enabling adaptive planning of optimized task execution sequences for each USV. The primary objective is to minimize the average waiting time and total time required for USVs

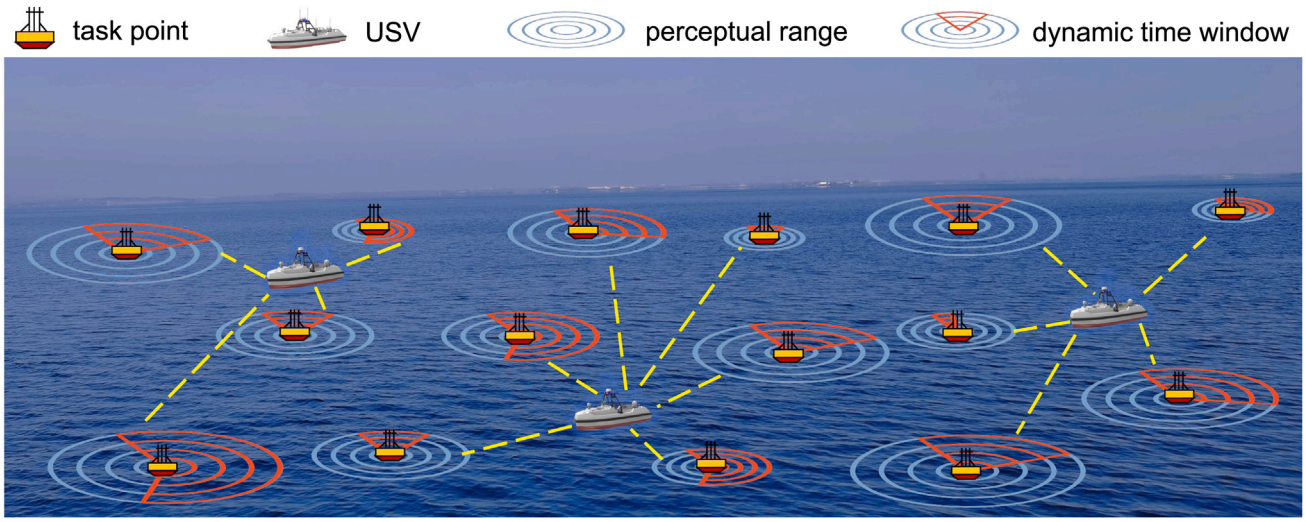


Fig. 1. Scenario diagram of multiple USVs executing multi-constrained task points. (Perceptual distance constraint is shown by blue concentric circles and time window constraint is indicated by the ratio of red to blue sectors encircling the task point.)

to complete all tasks. Considering the physical reality that task points are devices with a certain perceptual range, the MCTP-ASOM algorithm incorporates the perceptual distance constraint of task points. Through this incorporation, the algorithm adaptively identifies the optimized execution positions for task points, effectively reducing the navigation costs linked to task execution.

The remainder of this paper is organized as follows. After this introduction, statements of the task decision-making problem for multiple USVs and the bi-layer task decision-making framework are described in Section 2. Section 3 offers a comprehensive introduction to the innovative MCTP-ASOM algorithm, proposed to tackle multi-constrained task planning challenges, followed by an elucidation of the multi-task allocation method for USV clusters. Finally, Section 5 concludes the paper by summarizing its contributions and outlining potential avenues for future research.

2. Problem formulation and task decision-making framework

2.1. Problem statement

In order to achieve precise observations of the target sea area, a substantial number of smart buoys should be deployed, which are characterized as devices with dynamic time windows and perceptual distance constraints. This paper focuses on the problem of information-gathering on a given number of task points (buoys) in a given closed sea area by utilizing USV clusters. The objective of this study is to plan k task execution paths, denoted as $Path_j = \{p_1, p_2, p_3, \dots, p_{m_j}\}$, for k USVs within a cluster, where $USV = \{usv_1, usv_2, \dots, usv_k\}$. These paths are designed to facilitate the execution of n task points $X = \{x_1, x_2, \dots, x_n\}$ by the USV clusters. Considering the physical model of task points as devices with a certain perceptual range. It is assumed that each task x_i has a perceptual radius r_{x_i}, forming a circular sensing area as shown in Fig. 2. The perceptual distance is the distance from the actual position of the task point to its perceptual boundary, which is equivalent to its perceptual radius in the absence of obstacles, and the USV only needs to reach any position within the perceptual area of a task point to collect information from that target.

Taking into account the dynamic time window constraint of the tasks, each task x_i carries three types of information: its position, the dynamic time window, and the perceptual radius. The perceptual radius and dynamic time window vary for each task point. Some key time-related information during the task planning process includes as follows and the relationship between the timing of the various time points is shown in Fig. 3.

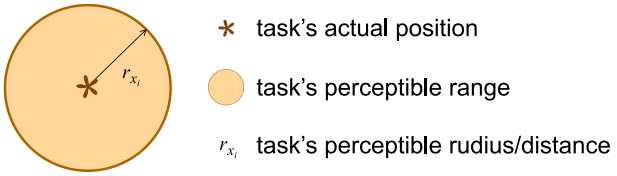


Fig. 2. The perceptual distance constraint of task points.

- $T_{x_i}^{window}$: a time window of task point x_i consists of a buffer time and an active time together, as shown in Fig. 3. During the buffer time task point x_i is blocked and cannot be executed. During the active time, the task can be executed if USV reaches the perceptible radius of the task point.

Some of the key timing information during the execution of task x_i by usv_j is described below.

- $t_{x_i}^{arrive}$: represents the moment when the usv_j enters the perceptible range of task point x_i.
- $t_{x_i}^{start}$: represents the moment when usv_j can start executing task x_i.
- $t_{x_i}^{end}$: represents the moment when usv_j accomplished task x_i.
- $T_{x_i}^{navigation}$: the time required for the usv_j to travel from the previous task point x_{i-1} to task point x_i, which is determined in this paper by dividing the required navigation distance by the speed of the usv_j, and equivalent to $\left(t_{x_i}^{arrive} - t_{x_{i-1}}^{end}\right)$.
- $T_{x_i}^{wait}$: when usv_j enters the perceptible range of the task point x_i, if task x_i is on buffer period, $T_{x_i}^{wait}$ is the waiting time required for x_i to pass through the current buffer phase.
- $T_{x_i}^{execution}$: represents the length of time required to perform task point x_i.

Denoting that the number of tasks assigned to usv_j is num_{usv_j}, the total number of tasks $n = \sum_{j=1}^{k} num_{usv_j}$, the cost of navigation for usv_j to perform assigned tasks is calculated as Eq. (1).

$$cost_{usv_j} = \sum_{i=2}^{m_j} \|p_{i-1} - p_i\| + \left\|p_{m_j} - p_1\right\|. \tag{1}$$

where m_j represents the number of path points for usv_j to complete all assigned tasks, p_{i-1} and p_i are the adjacent path points during the

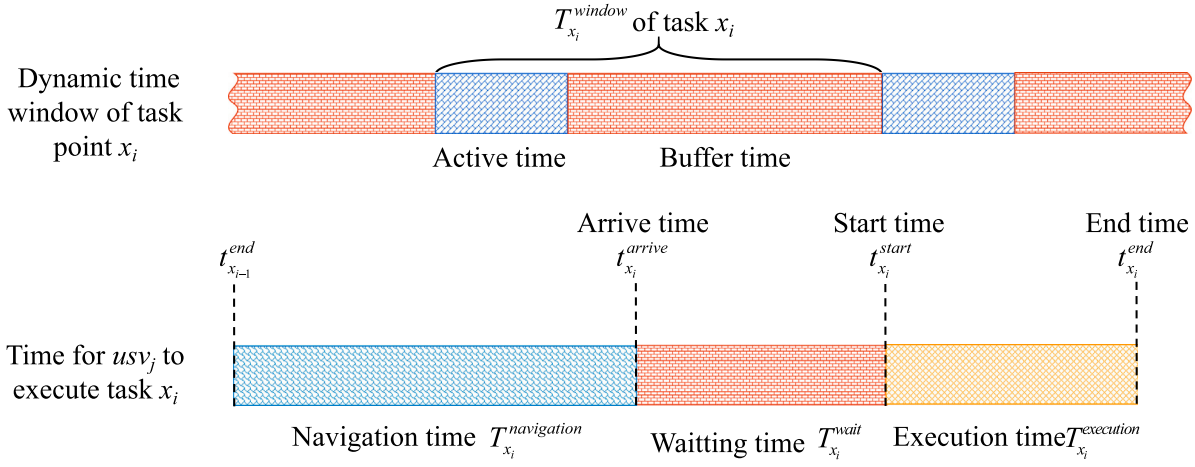


Fig. 3. The relationship between the timing of the various time points.

execution of the assigned task by usv_j. $\|p_{i-1} - p_i\|$ denotes the Euclidean distance between them. The time required for usv_j to perform assigned tasks is calculated as Eq. (2).

$$t_{usv_j} = \sum_{i=1}^{num_{usv_j}} T_{x_i}^{wait} + \frac{cost_{usv_j}}{v_{usv_j}}. \tag{2}$$

In Eq. (2), $\sum_{i=1}^{num_{usv_j}} T_{x_i}^{wait}$ represents the sum of waiting times required for usv_j to complete all assigned tasks, and v_{usv_j} denotes the traveling speed of usv_j. The cost required for the cluster to execute all tasks is calculated as Eq. (3) and the time required to execute all tasks is calculated as Eq. (4).

$$COST = \sum_{j=1}^{k} cost_{usv_j}. \tag{3}$$

$$T = \max\{t_{usv_1}, t_{usv_2}, \ldots, t_{usv_k}\}. \tag{4}$$

One of the solution objectives in this paper is to minimize the cost $COST$ required for the USV cluster to complete tasks. Considering the dynamic time window constraint associated with the tasks, the second objective function in this paper aims to minimize the shortest average waiting time for the USV to execute each task, denoted as $\min\left(\sum_{i=1}^{n} T_{x_i}^{wait}\right)$. Thus, the time T for the USV cluster to complete all tasks is minimized.

To simplify the problem model, this paper proposes the following basic assumptions and constraints:

- The information regarding the tasks is pre-generated and known, each task has the same priority and is located in the task area (Assumption 1).
- Information collection is considered an instantaneous operation, the time required by the USV to perform the single task is extremely short and negligible (Assumption 2).
- The USVs within the cluster are homogeneous, with constant power for all (Assumption 3).
- Communication between each USV is idealized (Assumption 4).
- To simplify the problem solution complexity, it is assumed that the active time of the task point is extremely short, simplified to 0 (Assumption 5).
- Each task is visited at least once and only once (Constraint 1).
- Each USV must depart from and return to the same position (Constraint 2).

2.2. A bi-layer framework for task decision-making

Due to the NP-complete nature of the task decision-making problem for USV clusters, achieving an optimal solution within polynomial time in practical applications is challenging. Consequently, a pragmatic trade-off between time efficiency and optimality is necessary to derive satisfactory solutions within a constrained timeframe. This paper presents an innovative solution by formulating the multi-objective task decision-making problem for USV clusters as a bi-layer optimization challenge, introducing a bi-layer task decision-making framework. The framework decomposes the multi-USV, multi-task decision-making into two distinct sub-problems: task allocation and task planning. The up-layer utilizes the K-means algorithm to achieve multi-task allocation for multi-USV, aiming to minimize the overall task execution cost. Meanwhile, the low-layer (task planning) utilizes the MCTP-ASOM algorithm to optimize the task execution sequence and determine optimized execution positions. The decision outcomes yielded by this approach are both rational and concise, effectively minimizing the overall cost of task execution for the USV cluster. The bi-layer framework is illustrated in Fig. 4.

In the up-layer, the multi-task allocation phase assumes a cluster comprising k functionally identical USVs tasked with performing n independent tasks of the same type, with full task information available. The objective is to allocate USVs within the cluster to execute a specific number of tasks, aiming to minimize the overall task execution cost while ensuring the completion of all tasks. Leveraging the well-established semi-supervised K-means algorithm (Sinaga and Yang, 2020), the approach achieves multi-task allocation for the multiple USVs. The goal is to determine a set of tasks for each USV to visit, ensuring that each task belongs exclusively to one task set. Once the allocation is finalized, the scheduling of USVs in the cluster follows their assigned task sets. Given the proximity of tasks, this scheduling strategy minimizes the total distance traveled by the USVs, consequently reducing the overall cost of task execution to the greatest extent possible.

The low-layer functions as the cluster task planning layer, with a primary focus on planning the execution paths for each USV within a cluster assigned a specific task set. The objective is to accommodate variations in characteristics and constraints among task points, including diverse dynamic time windows and perceptual distance constraints. The paper treats the task execution planning problem as a TSP (Miller et al., 1960), and considering the dynamic time window constraint of tasks, the MCTP-ASOM algorithm determines the optimized task execution sequence for each USV, minimizing the average waiting time for task completion. Notably, the introduction of the perceptual distance constraint allows for more accurate decision-making on the optimized execution position of task points by considering the perceptual range of task points, this effectively reduces the navigation cost associated with task execution. Concurrently, to ensure path safety, the paper incorporates the Fast Marching Squares (FMS) algorithm (Liu

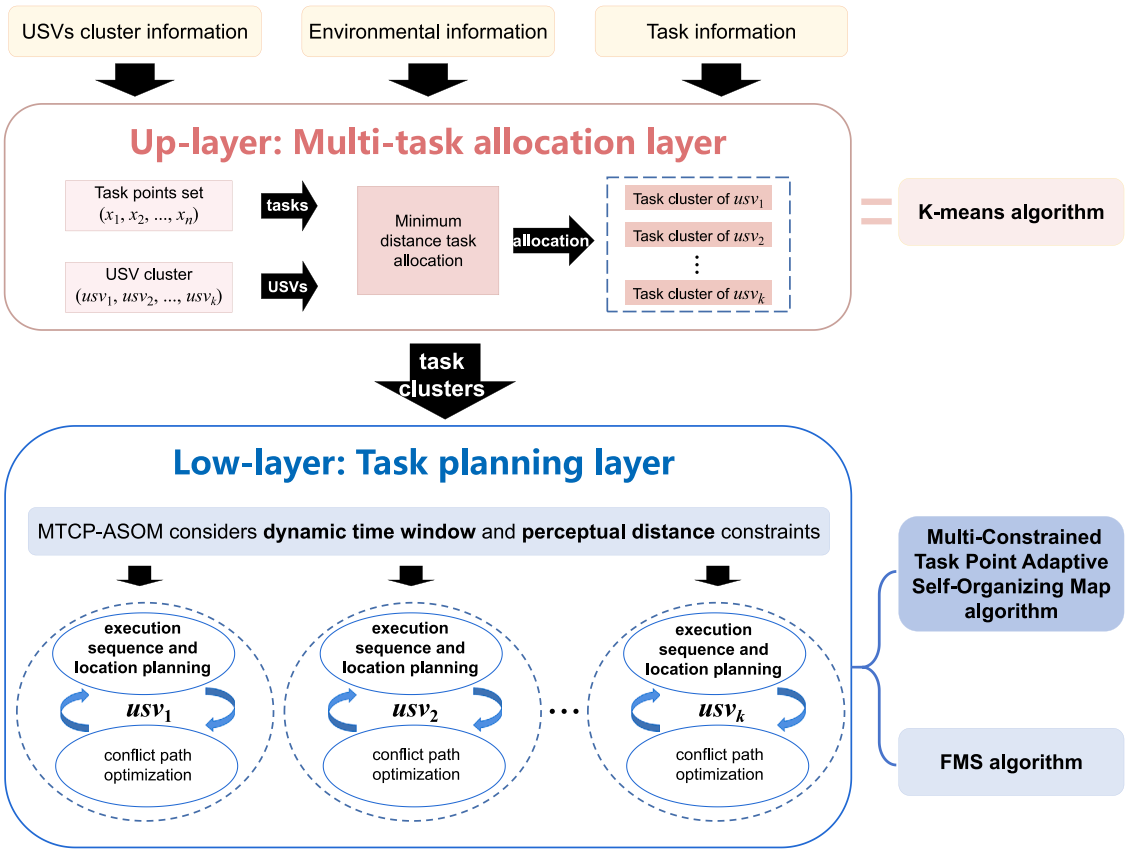


Fig. 4. Schematic of the bi-layer task decision-making structure for USV clusters.

and Bucknall, 2018) to optimize conflict paths. This approach aids in avoiding prohibited areas, ensuring that USVs steer clear of dangerous zones during task execution. As a result, the proposed method enhances the reliability and safety of the execution process.

3. The task decision-making approach for multi-constrained task points

3.1. Task planning with multi-constrained task points using MCTP-ASOM algorithm

The primary objective of the Low-layer module is to strategically plan suitable execution paths for all the USVs in the USV cluster that have been given the set of assigned tasks, meticulously considering the dynamic time window and perceptual distance constraints associated with these tasks. In this paper, the task planning challenge for each USV is framed as the TSP, a prototypical NP-hard problem inherently incapable of optimal polynomial time solutions. With $(n-1)!/2$ possible paths for n tasks, the exponential growth in solutions with an increasing n results in a computational complexity proportional to $n!/2$. While existing methods perform admirably for a small number of targets, scalability challenges arise with a larger task count, leading to computationally intensive and time-consuming processes, often yielding suboptimal outcomes (Patten et al., 2015). Considering the inherent constraints of tasks will significantly increase the planning complexity. The SOM neural network, an unsupervised learning model (Kohonen, 1990), facilitates the learning process by simulating competition and inhibition among neurons. This model preserves the spatial layout of the input space through a neighborhood relationship function, showcasing self-organization. Comprising input and output layers (Kohonen, 1982), the SOM neural network, when employed for task planning, represents each node in the input layer as a task point x_i, with weights corresponding to the positions of neurons in the output layer along the same dimension. A ring topology is established through competition, with neurons vying for victory. The structural depiction of the SOM neural network is presented in Fig. 5.

Throughout the training process, the SOM neural network has two learning rules. **Winner Take All (WTA) rule:** For a given input X, neurons in the output layer of the network engage in competition. This process ensures that only one neuron is in an excitatory state at any given time, while the others are in an inhibitory state. The neuron in the excitatory state termed the winner neuron or best matching unit (BMU), is the one most closely associated with the selected input task point. **Kohonen's rule:** Following the identification of the winning node (BMU), this node updates its vector weights to bring the BMU closer to the task points in the input space. This update process influences the state of the neurons surrounding the BMU. The impact of a BMU neuron on its neighboring neurons transitions from excitatory to inhibitory as the distance between the neurons increases.

The formulation of effective competition and neighborhood update functions is paramount in implementing the WTA rule and Kohonen's rule within a SOM neural network. This paper introduces an enhanced MCTP-ASOM algorithm specifically designed to tackle the task planning problem involving multi-constrained task points. In contrast to the conventional SOM algorithm, the MCTP-ASOM algorithm incorporates redesigned competition and neighborhood update functions to optimize its performance.

3.1.1. Competition function design

In traditional SOM algorithm applications for task planning, the competition function design hinges solely on the Euclidean distance between neurons and task points. To enhance the suitability for planning multi-constrained task points, the proposed MCTP-ASOM algorithm introduces an innovative dual competition mechanism. The competition

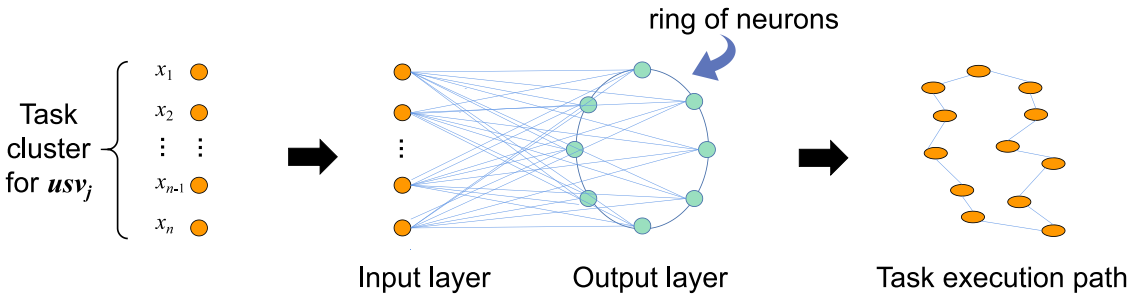


Fig. 5. Schematic of SOM neural network with two-layer structure for task sequence planning.

between neurons is defined by the following formula:

$$w_{BMU} = \underset{w_j \in W}{\operatorname{argmin}} \left(Dis_j + c \times I_{w_j} \right). \tag{5}$$

Among them,

$$Dis_j = \left\| x - w_j \right\|. \tag{6}$$

Dis_j denotes the Euclidean distance between the current task point and all output neurons. The coefficient c governs the influence of waiting time on the selection of the BMU, where a higher value of c indicates a more pronounced inhibition of waiting time in the BMU selection process. During the initial stages of training, a smaller value of the parameter c is employed to mitigate the impact of neuron waiting times on the BMU selection. This approach increases the likelihood of neurons in closer proximity being selected as the BMU. Such a strategy expedites the establishment of a relatively stable relationship between neurons and task points, emphasizing the swift formation of neural topology in the initial phase. As training progresses through subsequent iterations, the value of c is gradually increased. This adjustment, correlated with the number of iterations, enhances the chances of neurons with shorter waiting times competing successfully as the BMU. This strategy aids in fine-tuning the arrangement of neurons, contributing to the reduction of overall waiting time for task completion. During this phase, greater emphasis is placed on adjusting the local structure of neurons. Before the first iteration, the task execution path $Path$ is randomly initialized ordering of task points. In each subsequent iteration, the current task execution path $Path$ is updated based on the result of the previous iteration. The arrival time t_x^{arrive} and waiting time T_x^{wait} required for executing the task are then calculated and updated according to the current $Path$ and the dynamic time window constraints of the tasks. The waiting time of the nearest task point to the current neuron has an inhibitory effect on its competition for the winning neuron, the longer the waiting time, the greater the inhibitory effect. Thereby, the neuron prefers to run to the task point with a shorter waiting time. For example, if x_i is the nearest target task point to the neuron w_j, the inhibition effect factor I_{w_j} for the current neuron w_j is calculated as follows, where λ is a coefficient that adjusts the influence of task waiting time on the inhibition of neurons:

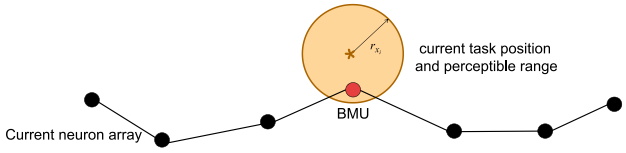
$$T_{x_i}^{wait} = T_{x_i}^{window} - t_{x_i}^{arrive} \% T_{x_i}^{window}. \tag{7}$$

$$I_{w_j} = \lambda \times t_{x_i}^{arrive}. \tag{8}$$

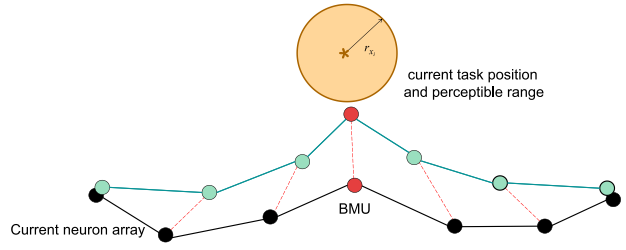
This approach helps neurons favor task points with shorter waiting times during the competition process.

3.1.2. Neighborhood function design

Considering the distance constraint of task points during the iteration process leads to two distinct scenarios. In the first case, if the elected BMU is already within the perceptible range of the current task point—meaning the distance between the BMU and the current task point is less than or equal to the perceptible radius of the current task point—then the current BMU position is deemed the optimized execution position. In this scenario, there is no necessity to execute the neural network iteration process, as illustrated in Fig. 6(a). Conversely, the second case arises when the BMU lies outside the perceptible range of the current task point. In such instances, the weights of the neuron and its surrounding neurons need to be updated systematically. This updating process guides the neurons gradually toward the optimized task execution position, as depicted in Fig. 6(b).



(a) The distance between the BMU and the current task point is less than or equal to the perceptible radius of the current task point.



(b) The BMU is outside the perceptible range of the current task point.

Fig. 6. Illustration of the BMU position relative to the perceptual range of the current task point.

Compared with the SOM algorithm, this paper adds the dynamic time window constraint of the task point to the neighbor relationship function. For each output neuron w_j, the neighborhood function update strategy is designed as follows:

$$w_j = w_j + f(d, iter) \times \left(\alpha \times \left\| x_i - w_j \right\| - \beta \times T_{x_i}^{wait} \right). \tag{9}$$

$T_{x_i}^{wait}$ is the waiting time of the current task point, calculated according to Eq. (7). $f(d, iter)$ is a neighborhood function to adjust the direction and strength of the neighborhood update. The α and β are learning rate parameters that decrease with iterations, controlling the magnitude of neighborhood updates.

$$\alpha_{iter+1} = \alpha_{iter} \times 0.9. \tag{10}$$

$$\beta_{iter+1} = \beta_{iter} \times 0.9. \tag{11}$$

$$c_{iter+1} = c_{iter} \times 1.01. \tag{12}$$

The function $f(d, iter)$ serves as a neighborhood function, and its coverage influences both the BMU and neighboring neurons. Nodes closer to the BMU possess higher weights, while nodes farther away from the BMU have lower weights. If the influence of the BMU is expressed as [0,1], the closer the intensity is to 1, the more influence its neighboring neurons will have and the greater the distance they will move toward the task. Conversely, as the intensity approaches 0, the influence on neighboring neurons diminishes, and it may even cease entirely. When the influence drops to 0, the neuron is no longer considered a neighboring neuron of the BMU and is referred to as a non-neighboring neuron. The schematic of the neighborhood update process is shown in Fig. 6(b).

In this paper the neighborhood function is denoted as follows:

$$f(\mathrm{d}, iter) = \frac{1}{\sqrt{2}} \cdot e^{-\frac{d^2}{G_{iter}^2}}. \tag{13}$$

where d denotes the distance between the neuron to be updated and the BMU, $iter$ is the number of rounds of iteration after each iteration, G_{iter} is based on Eq. (14) decreases gradually, and ε in Eq. (14) is the gain reduction ratio.

$$G_{iter+1} = (1-\varepsilon) \times G_{iter}. \tag{14}$$

The G-value's role in neural network training is pivotal. Initially, it enables the BMU to significantly influence distant neurons in the input layer, promoting rapid stabilization. As training progresses and neuron competition wanes, the G-value diminishes, leading to a shrinking BMU neighborhood. This refinement consolidates stability and enhances the network's ability to determine subsequent BMUs. The adaptive neighborhood update strategy ensures neurons evolve in a direction that aligns with task planning demands, particularly within dynamic time constraints.

3.1.3. Neuron additions and deletions

When employing the SOM algorithm for task planning, each neuron signifies a potential task point. Minor adjustments among the original neurons may cause the algorithm to converge to a local optimal solution, potentially missing the global optimal solution. To mitigate this, it becomes imperative to incorporate neuron addition and deletion operations to broaden the exploration scope of the algorithm. The add-neuron operation augments the number of neurons as the iteration progresses, thereby expanding the search space. Conversely, the remove-neuron operation eliminates redundant neurons, reducing the depth of the search space and accelerating algorithm convergence.

The improved MCTP-ASOM algorithm records the number of iteration rounds in which a neuron was last selected as a BMU by any task point and the number of times it competed to become a BMU throughout the iterations. If a neuron fails to be selected as a BMU by any task point in three consecutive iterations, the neuron is deleted. The neuron count is reduced by 1, and associated information such as counter values and iteration times are purged. At regular intervals, the neuron with the highest frequency of BMU selection is chosen, and a new neuron is inserted in the middle of the neuron with the greatest distance neighbor. Subsequently, counter values, last iteration times, and other information for all neurons are updated. Following each round of iterations, adjustments are made to the number of neurons, as well as the learning rate α and other parameters, gradually optimizing algorithm performance. The entire algorithm operates within a predefined maximum iteration limit, and upon convergence, each task aligns with the corresponding neuron while preserving its initialized topology, resulting in a more adaptive neuron arrangement W. The sequence of waypoints $Path$, representing the execution task sequence and positions for each USV, is derived from the output neurons of the MCTP-ASOM. The task scenario presented in this paper simulates a real marine environment. Irregular obstacles are randomly distributed throughout the task area, emulating the islands and reefs found in actual maritime settings. Consequently, the task execution path devised by the MCTP-ASOM algorithm may pass through these obstacle regions. To ensure the safety and feasibility of the path, it is essential to optimize the route that traverses the obstacles. To address this, this paper incorporates the FMS algorithm to rectify conflicting paths. FMS offers continuously smooth paths and allows for the adjustment of the safety distance between the planned path and obstacles, enhancing adaptability to marine environments. The corrected paths will be updated to the waypoint sequence $Path$.

The proposed MCTP-ASOM algorithm for task planning with perceptual distance and dynamic time window constraints is outlined in Algorithm 1.

Algorithm 1: Task planning based on MCTP-ASOM algorithm

Input: The set $\{x_1, x_2, ..., x_n\}$ carrying task information, k clusters $Clusters = (Cluster_1, Cluster_2, ..., Cluster_k)$ denote task set assigned to each *usv*, maximum number of iterations $iter_{max}$

Output: Task execution Path $Path$

for $Cluster_l \in Clusters$ **do**
  $X \leftarrow$ Extract the set of tasks for usv_l from $Clusters_l$
  INITIALIZATION
  $NeuronsNum \leftarrow multi \cdot |X|$
  $|W| \leftarrow NeuronsNum$ `// Randomly initialize output neurons` $W \subseteq \mathbb{R}^2$
  $Path \leftarrow$ Randomly initialize ordering of task point
  for $iter = 1, 2, ..., iter_{max}$ **do**
    for each $w_j \in \mathbf{W}$ **do**
      $Dist_{x_i} = \| x_i - w_j \|$
      $I_{w_j} = \lambda \times T^{wait}_{x_{min(Dist)}}$
    end
    for each $x_i \in X$ **do**
      SELECTING BMU
      $Dist_j = \| x_i - w_j \|$
      $w_{BMU} = min(Dist_j + c \times I_{w_j})$
      ADDING AND DELETING NEURONS
      add-neuron operation;
      remove-neuron operation;
      NEIGHBORHOOD UPDATING
      for each $w_j \in \mathbf{W}$ **do**
        $w_j = w_j + f(d,t) \times \left(\alpha \times \| x_i - w_j \| - \beta \times T^{wait}_{x_i}\right)$
      end
    end
    $\alpha, \beta, c \leftarrow Update(\alpha, \beta, c)$
    $Path_l \leftarrow Update\left(Path_l, W\right)$
  end
end

Task planning is the implementation of low-layer in this paper. Preceding the task planning for each USV, it is imperative to obtain the task set for each USV. This crucial step is facilitated through task allocation in the upper layer, and the specific details of the task allocation implementation are provided in Section 3.2.

3.2. Multi-task allocation for USV clusters based on K-means algorithm

In this study, the aim is to effectively allocate a given set of tasks to different USVs. K-means is one of the simplest unsupervised learning algorithms to solve the well-known clustering problem. It is a mature clustering algorithm based on minimizing the distance to centroids. It can be used to group multiple tasks, and after clustering, tasks

within the same cluster will have relatively short distances between them. Subsequently, specific cluster tasks can be assigned to relevant USVs, minimizing the travel distance of the USVs. The process of task allocation using the K-means algorithm follows a straightforward approach, classifying the given dataset by pre-setting a fixed number of clusters. The specific implementation steps are as follows:

1. The initial centroid $Centroids$ is generated by randomly selecting k sets of coordinates from the task set X.
2. Assign each target task x_i to the centroid element $Centroid_j$ in the set of $Centroids$ with the minimum Euclidean distance until all task points have been assigned. As shown in the following:

$$Cluster_{assign} = \underset{Centroid_j \in Centroids}{\operatorname{argmin}} \left\| Centroid_j - x_i \right\|. \tag{15}$$

   Such that every single task point will have an assigned centroid and the task points assigned to the same $Centroid_j$ form a cluster $Cluster_j$.
3. The arithmetic mean of the elements within $Cluster_j$ is calculated and then used to update $Centroid_j$, representing the new centroid element.

$$Centroid_j = \frac{1}{\left| Cluster_j \right|} \sum_{x_i \in Cluster_j} x_i. \tag{16}$$

   $\left| Cluster_j \right|$ denotes the number of task points assigned to $Cluster_j$, and $Centroid_j$ is the mean of $Cluster_j$.
4. Repeat steps (2) and (3), the centroid gradually changes the position for iteration until the clustering results no longer change, and clustering is complete.

In the process, the algorithm aims to minimize the value of the objective function:

$$\underset{Cluster}{\operatorname{argmin}} \sum_{j=1}^{k} \sum_{x_i \in Cluster_j} \left\| x_i - Centroid_j \right\|. \tag{17}$$

where $Cluster$ denotes the sequence containing k clusters $(Cluster_1, Cluster_2, \ldots, Cluster_k)$. $\left\| x_i - Centroid_j \right\|$ is the Euclidean distance between the target point x_i and the cluster center $Centroid_j$. When an updating process is involved, the existing centroid is updated to the average of the tasks belonging to its corresponding cluster.

After multiple iterations, the task set assigned to each USV is extracted from the $Cluster$. The implementation of the K-means algorithm for multi-objective task allocation is represented in Algorithm 2.

Algorithm 2: Multi-task allocation based on K-means algorithm

Input: 2-dimensional set of task coordinates $X = \{x_1, x_2, ..., x_n\}$,Number of USVs in cluster k,The maximum number of iterations $iter_{max}$

Output: $Cluster$

INITIALIZATION
$Centroids \leftarrow$ randomly selecting k coordinates from task set X
for $iter = 1, 2, ..., iter_{max}$ **do**
  TASK ALLOCATION
  for *all* $x_i \in X$ **do**
    $dist \leftarrow$ Euclidean distance between x_i and all $Centroids$
    $assign \leftarrow$ the index of $min(dist)$
    $Cluster_{assign} \leftarrow x_i \cup Cluster_{assign}$
  end
  UPDATING CENTROIDS
  for $j \leftarrow 1, 2, ..., k$ **do**
    $Centroids_j \leftarrow mean(Cluster_j)$
  end
end

Using the above theory, the specific decision-making process for USV clusters to perform the task of completing multi-constrained task points is shown in Fig. 7.

4. Case study

In this paper, a series of simulations were conducted to validate the effectiveness of the proposed approach. All simulations were developed on a 64-bit laptop featuring an Intel(R) Core(TM) i5-8300H CPU @ 2.30 GHz processor with 32 GB RAM, using a combination of C++ and MATLAB languages. The data from the simulation results were processed using MATLAB R2022a.

During these simulations, task positions were randomly distributed on the graph, while the positions of island areas were predetermined and represented by black pixels in the graph. To ensure task feasibility, a prerequisite was established that no tasks should be located within the island areas. Subsequently, the proposed task decision-making approach in this paper was evaluated in two cases: (1) For task points with only perceptual distance constraints, the up-layer employs the K-means algorithm to assign a precise task set to each USV. Following this, the MCTP-ASOM algorithm and the SOM algorithm are used with the FMS to plan the task execution paths for a single USV, respectively. This process not only demonstrates the effectiveness of the bi-layer task decision-making framework but also confirms the superior performance of the MCTP-ASOM algorithm over the SOM algorithm in reducing navigation costs for single-constrained task points. (2) For task points with multi-constraints, aiming to minimize the time taken for the cluster to complete tasks and the individual waiting time of each USV, the MTCP-ASOM algorithm is used to adaptively determine the task execution sequence and positions, thus validating the performance of the improved algorithm. Simultaneously, the performance of the SOM algorithm is compared with that of the MCTP-ASOM algorithm in various task scenarios.

Case study I: Task decision-making for multi-USV with perceptual distance-constrained task points

To address tasks with only perceptual distance constraints, the K-means algorithm is employed to assign tasks to individual USVs within the USV clusters. Subsequently, the MCTP-SOM algorithm during the task planning process considers only the perceptual distance constraint of the task points, and the FMS algorithm is used for obstacle avoidance for each USV, validating the performance of the proposed bi-layer framework structure. Meanwhile, the results of MCTP-ASOM are compared with the execution results of the SOM algorithm in the same task scenario.

The simulation rasterizes a 15 × 15 (n mile) map into a 500 × 500 raster map, with one raster equivalent to approximately 0.03 (n mile). A total of 100 tasks and some island areas are randomly distributed, and the specific distribution locations and perceptible ranges of the tasks are illustrated in Fig. 8. Details of the task points are provided in Table 1. The experiment involves sending a cluster of 7 USVs to execute the 100 tasks, with each task point reachable only once. Each USV in the cluster is isomorphic, with unlimited load and energy consumption, indicating no task load constraints. To assess the algorithm's robustness, the experiment includes overlapping areas between the perceivable range of some tasks and the island area, as exemplified by the task points circled by the green dotted line in Fig. 8.

In the task allocation layer, tasks assigned to different USVs are denoted by asterisks of distinct colors, and their perceptible areas are represented by circles around them. The outcome of task allocation using the K-means algorithm in the aforementioned scenario is depicted in Fig. 9. From Fig. 9, it can be seen that the K-means algorithm can successfully divide these 100 task points into 7 task sets and each task belongs to only one task set. That is, 100 tasks can be successfully assigned to these 7 USVs in the cluster. The number of tasks assigned to each USV is 16, 12, 14, 13, 15, 16, and 14 task points, respectively. For a given task set, the task points in the task set after grouping are

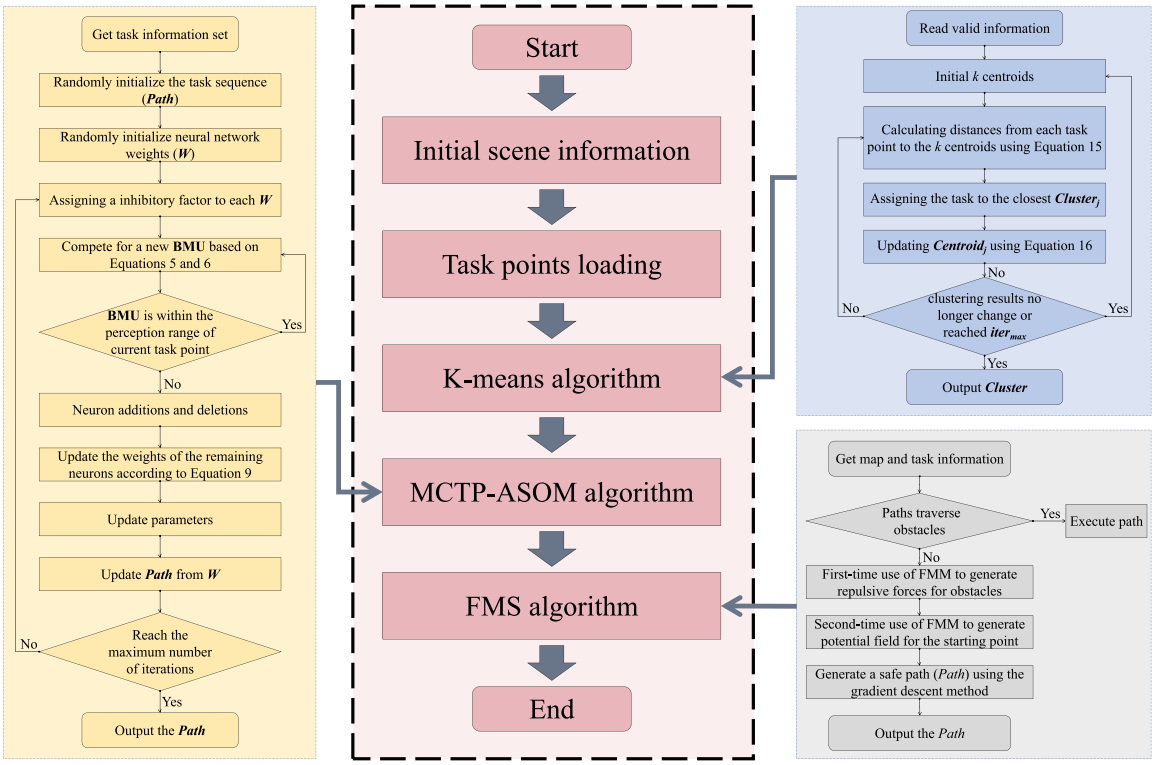


Fig. 7. Task decision-making flowchart for USV clusters with multi-constrained task points.

Table 1
Details of 100 multi-constrained task points.

id	x	y	r	T	id	x	y	r	T	id	x	y	r	T	id	x	y	r_n	T
1	431	142	12	13	26	323	140	12	7	51	432	85	9	7	76	282	155	11	8
2	480	400	11	6	27	105	200	15	14	52	376	100	15	12	77	389	192	13	10
3	399	57	9	8	28	475	109	15	11	53	286	282	15	15	78	365	469	12	14
4	288	5	12	15	29	30	365	13	11	54	203	109	12	6	79	11	405	8	10
5	130	241	11	7	30	126	21	9	8	55	333	205	9	11	80	255	120	9	12
6	163	95	9	7	31	317	494	9	13	56	14	160	12	8	81	274	361	14	10
7	389	421	14	13	32	444	237	9	13	57	302	181	9	15	82	77	40	11	11
8	47	326	9	15	33	239	59	15	11	58	321	330	13	13	83	283	412	9	8
9	108	122	10	7	34	41	407	8	5	59	411	86	9	9	84	2	476	14	13
10	254	476	11	11	35	491	164	11	10	60	500	32	14	7	85	70	175	9	10
11	343	410	11	14	36	382	25	13	12	61	222	236	11	6	86	448	477	13	12
12	470	431	10	13	37	19	281	10	6	62	198	277	15	7	87	314	91	12	6
13	72	253	12	12	38	200	460	9	8	63	178	134	10	7	88	364	288	10	11
14	429	331	15	5	39	163	42	12	14	64	12	225	9	14	89	238	403	12	7
15	70	432	11	9	40	247	215	11	8	65	235	440	15	11	90	355	75	13	11
16	494	468	15	7	41	346	441	15	5	66	60	464	12	14	91	115	92	9	6
17	49	193	12	11	42	498	299	10	14	67	208	22	8	15	92	137	420	8	15
18	489	247	11	15	43	391	257	12	8	68	322	53	13	9	93	152	263	14	14
19	114	285	15	12	44	456	37	14	12	69	43	108	9	7	94	71	348	15	15
20	134	139	8	14	45	400	373	9	11	70	306	449	12	11	95	36	245	13	5
21	430	203	10	5	46	86	398	10	9	71	457	181	13	11	96	425	499	8	10
22	292	77	8	11	47	247	178	14	7	72	361	340	10	15	97	5	322	14	14
23	144	181	13	14	48	166	230	14	9	73	37	26	10	14	98	377	149	12	6
24	352	372	15	7	49	341	256	13	15	74	280	233	11	13	99	416	465	9	14
25	353	124	8	10	50	429	411	12	15	75	309	397	9	9	100	194	192	10	14

Note: The "T" represents $T_{x_i}^{wait}$. x, and y denote the horizontal and vertical coordinates of the task point in the grid map respectively.

adjacent to each other and the distance between the task points is shorter, so the total distance traveled by the USVs will be shorter, which in turn will result in the shortest navigation time. This indicates that the proposed algorithm is effective in considering the task workload of each USV within the cluster during the task allocation phase and can generate relatively reasonable task allocation results.

Having obtained the set of tasks assigned to each USV, and considering that each USV is tasked with multiple assignments, the SOM algorithm is employed to plan task execution paths for each USV. The planning results are illustrated in Fig. 10. The ideal path for task execution is illustrated by a dotted line of the same color as the set of tasks. However, it is worth noting that the paths planned by the SOM may intersect with island areas during the USV's task execution, rendering the planned paths infeasible, as depicted by the paths in the chartreuse dotted circle in Fig. 10. Consequently, the FMS algorithm is applied to rectify the conflicting paths. The results of the corrected paths are shown in Fig. 11. The navigation distances planned by the SOM algorithm for each USV to execute the tasks after correcting

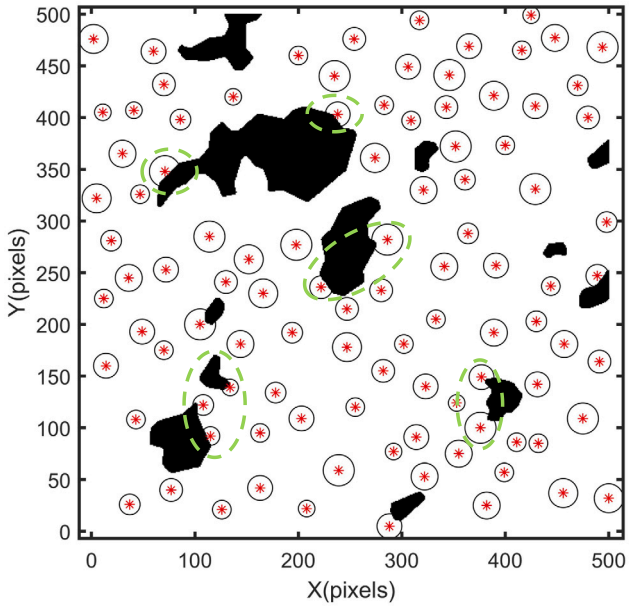


Fig. 8. Map of the distribution of the 100 randomized task points. (Red asterisk denotes the task's actual position, black hollow circle represents its perceptible range. Green dashed line shows the task's perceptible range intersects with obstacles.)

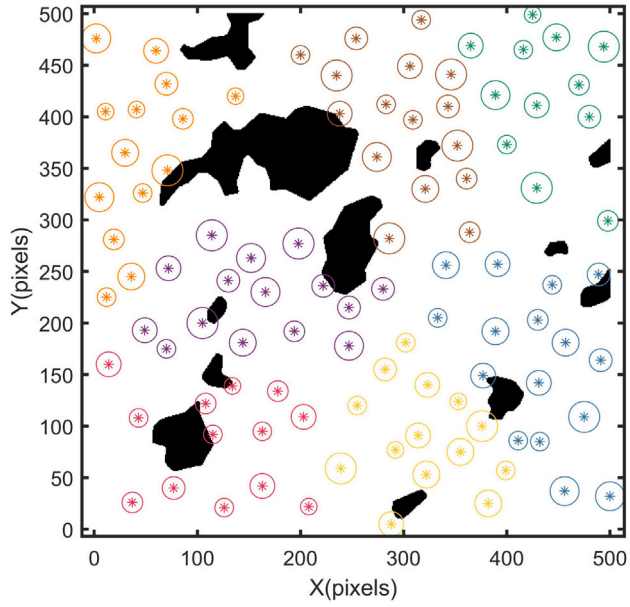


Fig. 9. The up-layer result assigns 100 task points to the 7 USVs in the cluster.

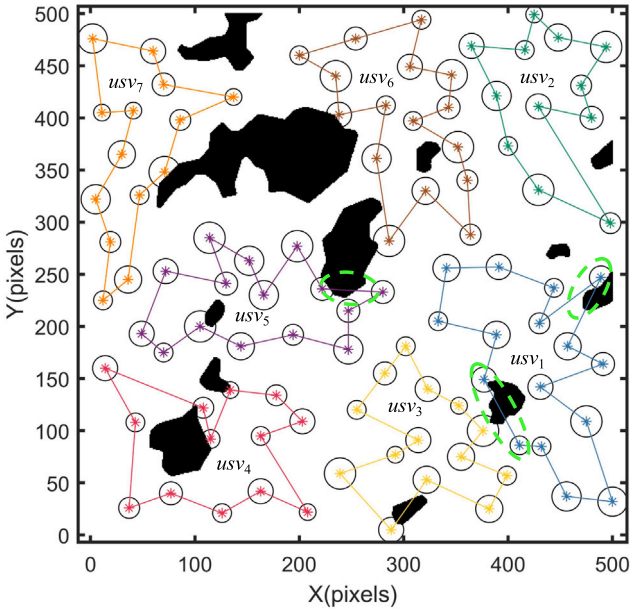


Fig. 10. Task planning result for each USV using the SOM algorithm. (The chartreuse dotted line inside the planned path overlaps with an obstacle and requires adjustment.)

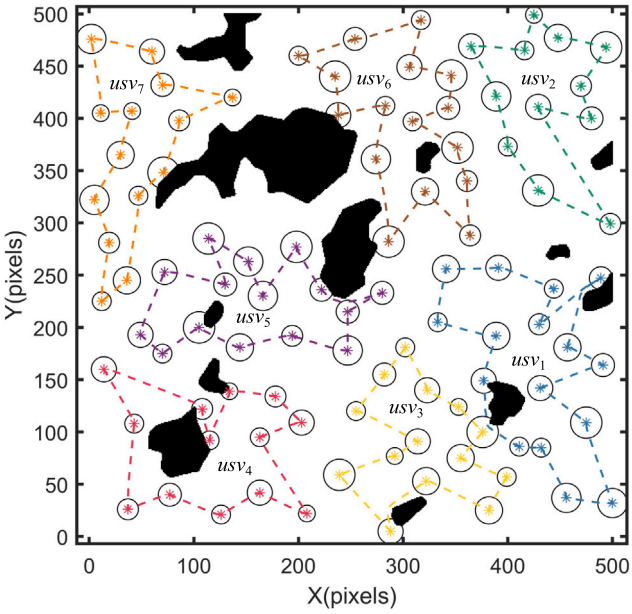


Fig. 11. Modified navigable task execution path planned by the SOM algorithm.

the paths are 25.18, 18.25, 18.493, 19.18, 20.07, 21.778, and 18.93 nautical miles, respectively. Figs. 10 and 11 illustrate that the traditional SOM algorithm does not account for the perceptual radius of task points when planning task execution paths. Consequently, the designated task execution point coincides with the task point's original location, which can lead to an increased navigation distance for the USV, and consequently, a higher navigation cost. In contrast, the MCTP-ASOM algorithm incorporates the perceptual radius constraint of task points into the task planning process. It identifies suitable execution positions at the periphery of the task's perceptual radius, rather than at the task point's actual location, which can effectively reduce the navigation cost during the task execution. The planning results of the MCTP-ASOM algorithm are presented in Fig. 12, and the path after correction is displayed in Fig. 13. After path correction, the MCTP-ASOM algorithm has planned the navigation distances for each USV to execute the tasks as follows: 22.17, 14.93, 16.26, 18.06, 16.52, 18.3, and 17.72 nautical miles, respectively.

Table 2 demonstrates that the navigation paths generated by the MCTP-ASOM algorithm are significantly shorter for each USV compared to those generated by the SOM algorithm. Specifically, reductions of 11.95%, 18.19%, 12.07%, 5.84%, 17.69%, 15.97%, and 6.39% are observed. These findings highlight the efficacy of the MCTP-ASOM algorithm in minimizing the navigation costs associated with USV task execution. Meanwhile, the above experiments demonstrate that the up-layer can allocate all tasks without conflicts to the USVs within the cluster, and the low-layer can plan feasible paths for each USV to execute all

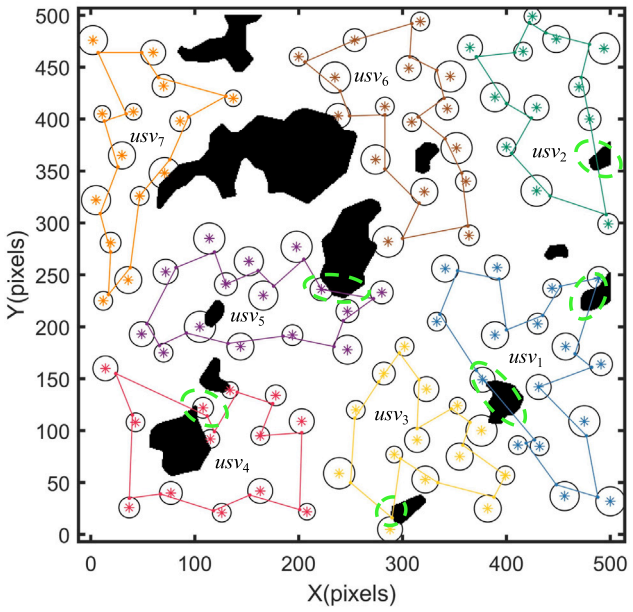


Fig. 12. Task execution paths planned for each USV using the **MCTP-ASOM** algorithm. (The chartreuse dotted line inside the planned path overlaps with an obstacle and requires modification.)

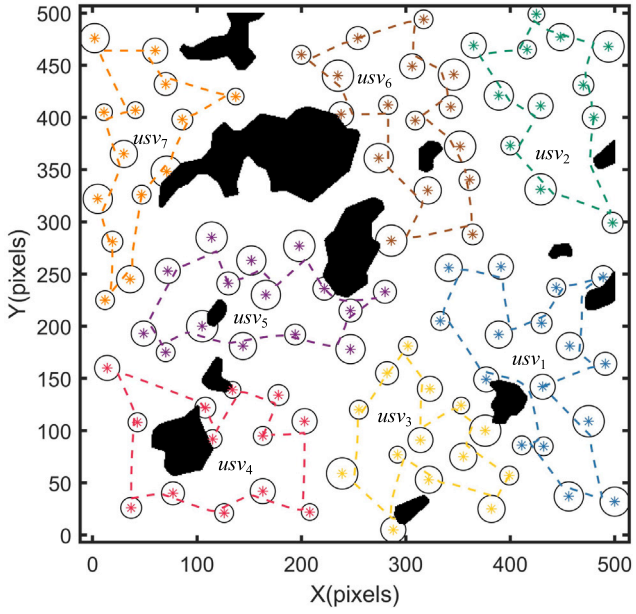


Fig. 13. Modified navigable task execution path planned by the **MCTP-ASOM** algorithm.

assigned tasks. Therefore, it can be concluded that the proposed bi-layer framework is effective in addressing task decision-making problems and reducing navigation costs.

Case study II: Task decision-making for multi-USV with multi-constrained task points

To address task points with dynamic time window constraints and perceptual distance constraints, aiming to minimize the completion time for the entire cluster of tasks and reduce the average waiting time for tasks, the K-means algorithm is employed to assign tasks to individual USVs within the USV clusters, the MCTP-ASOM algorithm is employed for task planning with multiple constrained task points for each USV. The FMS algorithm assists in avoiding obstacle areas

Table 2
Results from 7 USVs completing 100 task points.

USVs ID	Number of tasks	Navigation distance	
		MTSP-ASOM	SOM
usv_1	16	**22.17**	25.18
usv_2	12	**14.93**	18.25
usv_3	14	**16.26**	18.493
usv_4	13	**18.06**	19.18
usv_5	15	**16.52**	20.07
usv_6	16	**18.30**	21.778
usv_7	14	**17.72**	18.93

Note: The "navigation distance" is calculated according to Eq. (1), in nautical miles.

during the execution of tasks. Furthermore, the performance of the SOM algorithm is compared with the MCTP-ASOM algorithm across various task scenarios, substantiating the superior performance of the MCTP-ASOM algorithm.

Expanding on the task scenario in Case Study I, a dynamic time window constraint is introduced for each task point, as detailed in Table 1. This implies that each task point now has an active time and a buffer time. During the buffer period of a task point, even if a USV reaches the perceptual area of the task point, the task cannot be executed smoothly. Each USV in the cluster is isomorphic, with a constant speed of 30 knots and unlimited load and energy consumption. The allocation results, assigning 100 task points to these 7 USVs in the cluster, are presented in Fig. 9 and Table 2. Assuming that 7 USVs start executing tasks at the same time, each USV selects the smallest id task from the assigned task set as the starting position, i.e., it starts its task from the pentagram. The result after planning the task execution path for each USV using the MCTP-ASOM algorithm and correcting the conflicting paths is shown in Fig. 14(a). Each solid circle with color in the task circle represents the sector representing the ratio of the remaining buffer time to the total buffer time for the current turn of the task after the USV arrives at that task point, and the larger the sector area, the longer the waiting time for the USV to arrive at that task point. The task execution paths for each USV, planned using the SOM algorithm for multi-constraint task points, with conflict paths corrected using the FMS algorithm, are illustrated in Fig. 14(b). As shown in Table 3, compared to the SOM algorithm, MCTP-ASOM not only has excellent performance in reducing the navigation distance but also effectively reduces the total time for each USV to perform the assigned tasks with the average waiting time to perform each task. The maximum time-consuming USV for executing the tasks planned by the MCTP-ASOM algorithm is usv_5, which consumes 112 min, whereas the SOM algorithm plans the maximum elapsed time of executing tasks with usv_5 and 132 min, which is 20 min more than the MCTP-ASOM algorithm, thus showing that the proposed MCTP-ASOM algorithm in this paper can effectively reduce the maximum consuming time of executing the total tasks in the USV cluster.

There presents a comparative analysis of the MCTP-ASOM algorithm against the traditional SOM algorithm across various task scenarios. The evaluation criteria included time efficiency and navigation cost under conditions that featured different map layouts, obstacle configurations, task volumes, and USV cluster sizes. The simulation results detailed in Table 4, demonstrate the effectiveness of the MCTP-ASOM algorithm. In the first scenario, a cluster comprising 6 USVs was tasked with executing 50 randomly distributed task points. The up-layer effectively allocated the tasks without conflict to the USVs. Subsequently, the MCTP-ASOM and SOM algorithms were utilized in the low-layer to devise an execution path for each USV, with the respective results depicted in Figs. 15(a) and 15(b). The MCTP-ASOM algorithm outperformed the SOM algorithm by reducing the average waiting time per task point by 8.29%, the average waiting time per USV by 8.18%, the maximum task completion time for the cluster by 28.57%, and the total navigation distance by 16.99%. For the subsequent scenarios involving 5, 4, and 7 USVs tasked with 70, 80, and 100 random task

Table 3
The performance metrics of the MCTP-ASOM algorithm.

	Indicators	USVs ID						
		usv_1	usv_2	usv_3	usv_4	usv_5	usv_6	usv_7
	Number of tasks	16	12	14	13	15	16	14
SOM	Average wait time (min)	4.41	6.96	6.79	5.13	6.12	4.59	6.58
	Total wait time (min)	70.64	83.5	95.01	66.65	91.86	73.45	92.15
	Total execution time (min)	121	120	132	105	132	117	130
	Navigation distance (n miles)	25.18	18.25	18.493	19.18	20.07	21.778	18.93
MTSP-ASOM	Average wait time (min)	**4.10**	**6.18**	**3.18**	**2.30**	**5.26**	**3.71**	**3.90**
	Total wait time (min)	**65.67**	**74.12**	**44.48**	**29.86**	**78.95**	**59.41**	**54.55**
	Total execution time (min)	**110**	**104**	**77**	**66**	**112**	**96**	**90**
	Navigation distance (n miles)	**22.17**	**14.93**	**16.26**	**18.06**	**16.52**	**18.3**	**17.72**

Note: The "Total execution time" is calculated as Eq. (2).

Table 4
The experimental results comparing the SOM algorithm with the MCTP-ASOM algorithm across multiple task scenarios.

	Indicators	Task scenarios			
		Scenario 1	Scenario 2	Scenario 3	Scenario 4
	Number of tasks	50	70	80	100
	Number of USVs	6	5	4	7
SOM	Average waiting time per task (min)	4.34	5.78	4.8	5.73
	Average waiting time per USV (min)	36.18	80.97	98.59	81.89
	Total execution time (min)	91	140	162	132
	Total navigation distance (n mile)	71.45	85.06	123.88	141.87
MTSP-ASOM	Average waiting time per task (min)	**3.98**	**4.32**	**4.28**	**4.07**
	Average waiting time per USV (min)	**33.22**	**60.53**	**85.72**	**58.15**
	Total execution time (min)	**65**	**126**	**153**	**112**
	Total navigation distance (n mile)	**59.31**	**66.67**	**106.54**	**123.97**

Note: The "Total execution time" represents the maximum duration for each USV in the cluster to execute its tasks and is calculated as Eq. (4), with the time measured in minutes.

points, respectively, the MCTP-ASOM algorithm continued to exhibit significant improvements over the SOM algorithm. As illustrated in Figs. 16, 17, and 14, the MCTP-ASOM algorithm achieved reductions in the average waiting time per task point by 25.25%, 13.05%, and 28.97%, in the average waiting time per USV by 25.24%, 13.05%, and 28.99%, in the maximum task completion time for USV cluster by 10.00%, 5.56%, and 15.15%, and the total navigation distance by 21.62%, 13.98%, and 12.62%, respectively.

Simulations undertaken across a spectrum of scenarios have substantiated the effectiveness of the proposed approach, evidenced by an average reduction of 18.34% in wait time per task, an 18.86% decrease in wait time per USV, and a 16.3% reduction in overall navigation costs. The data presented provides strong evidence of the significant effectiveness of the bi-layer task decision-making approach. The approach demonstrates its ability to provide a reliable solution for clusters of USV to execute multi-constrained task points in a variety of scenarios while improving the efficiency and execution time of maritime tasks.

5. Conclusions and future work

Addressing the significant potential demonstrated by USV clusters in carrying out maritime information-gathering tasks, this paper introduces a bi-layer task decision-making framework and validates its feasibility through simulations. In response to the inherent constraints of task points in task planning, a novel Multi-Constrained Task Points Adaptive Self-Organizing Map (MCTP-ASOM) algorithm was introduced. This algorithm can adaptively plan the optimized task execution sequence for each USV while adjusting the optimized execution positions of task points. Extensive simulation demonstrates the algorithm's effectiveness in significantly reducing the maximum completion time of the USV clusters for all tasks, while reducing the average waiting time for task execution. The navigation cost required for USV to perform tasks is effectively reduced by adaptively finding the optimized execution location within the perceptual range of the task point rather than the location of the task point itself. In the future, we plan to further refine the multi-objective task planning algorithm to accommodate more complex environmental conditions and diverse task types. Simultaneously, we anticipate the application of the proposed algorithm in actual maritime environments, conducting more realistic and intricate field validations.

CRediT authorship contribution statement

Xiaoming Zhao: Writing – original draft, Software, Methodology, Data curation, Conceptualization. **Rui Song:** Writing – review & editing, Supervision, Methodology, Funding acquisition. **Yangmin Xie:** Resources, Project administration, Funding acquisition. **Zihao Wang:** Writing – review & editing. **Yan Peng:** Supervision, Resources, Conceptualization.

Declaration of competing interest

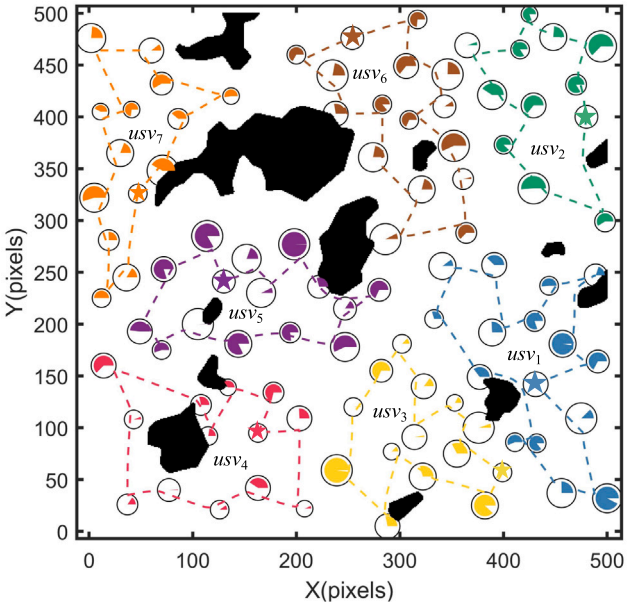
The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

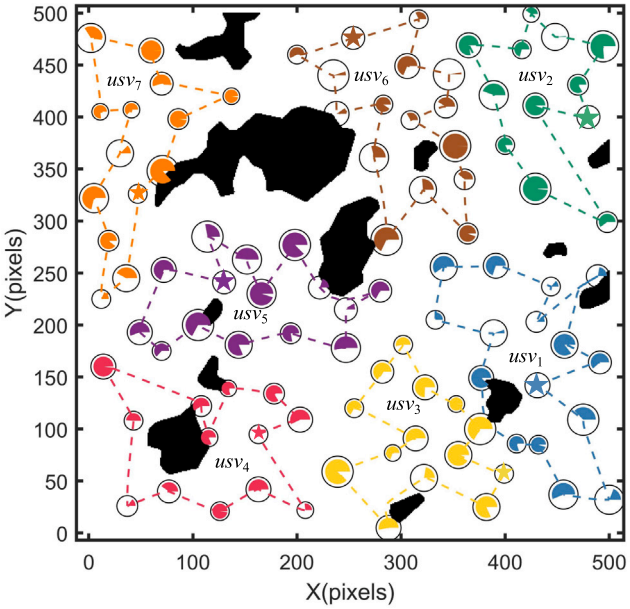
The authors are unable or have chosen not to specify which data has been used.

Acknowledgments

This research was funded by the National Key Research and Development Program of China, grant number 2021YFC2803003, and the Ministry of Science and Technology of the People's Republic of China with Grant No. 2021ZD0140300.

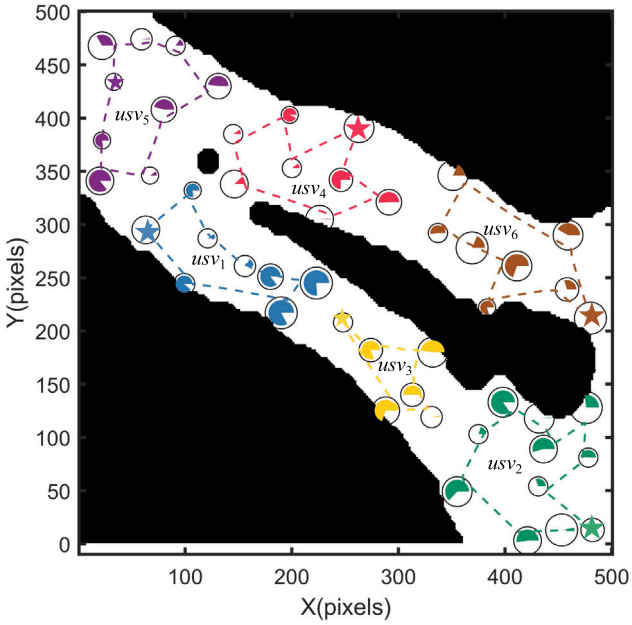


(a) Conflict path modified navigable path. (Coloured solid circle represents the wait time after the USV arrives at the task point. Wait time is proportional to the area of the colored sector.)

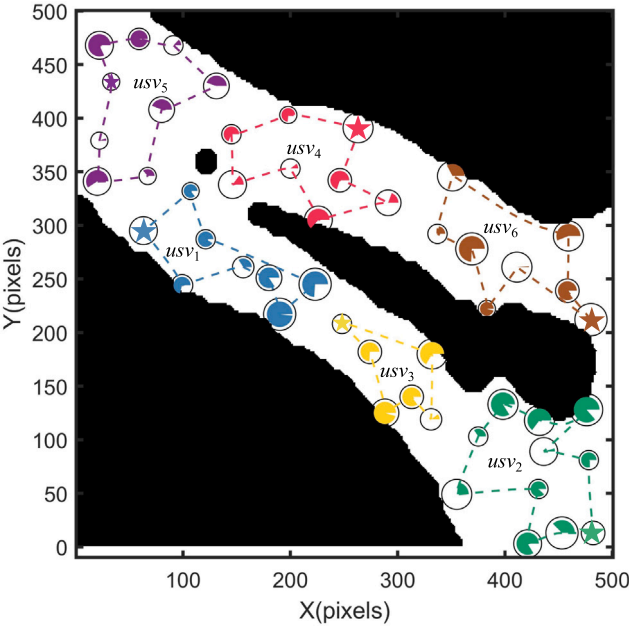


(b) Simulation results of path planning for 7 USVs using SOM algorithm at low-layer.

Fig. 14. Simulation results of bi-layer task planning for 7 USVs executing 100 multi-constrained task points.

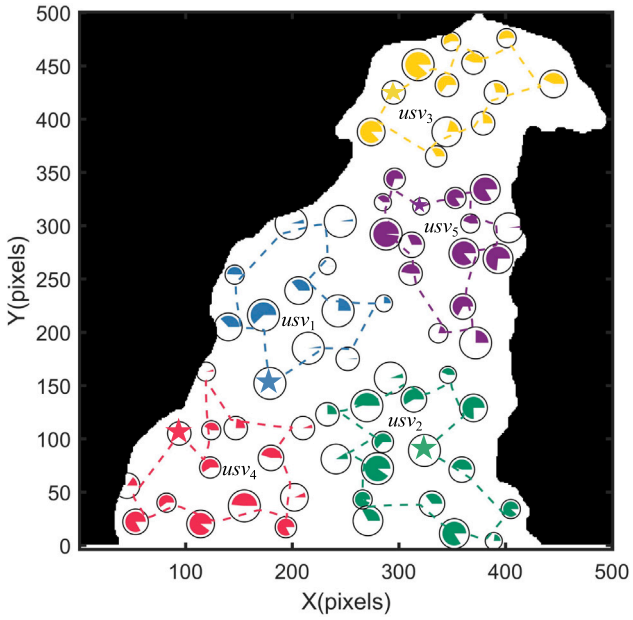


(a) Simulation results of path planning for 6 USVs using **MCTP-ASOM** algorithm at low-layer.

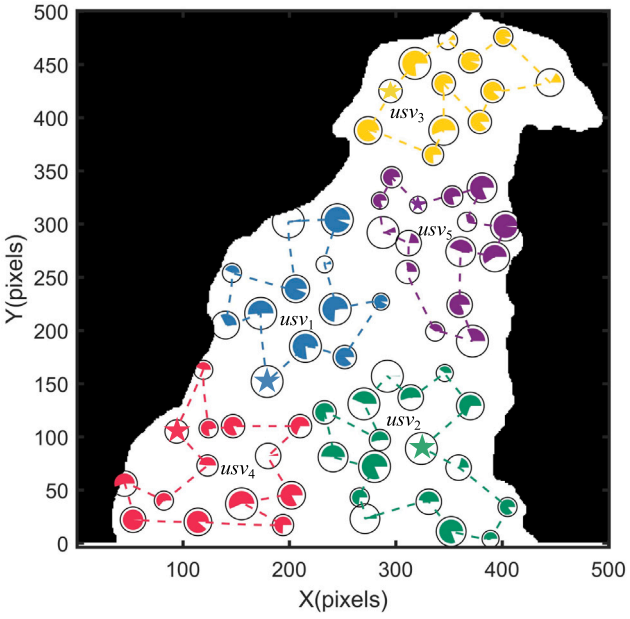


(b) Simulation results of path planning for 6 USVs using SOM algorithm at low-layer.

Fig. 15. Simulation results of bi-layer task planning for 6 USVs executing 50 multi-constrained task points.

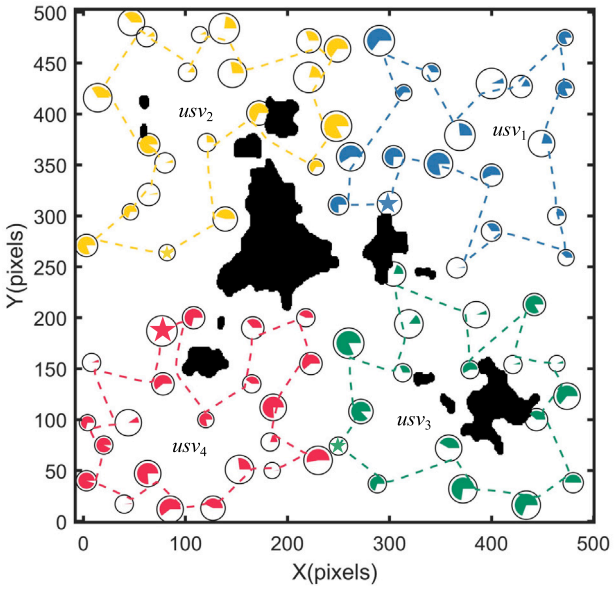


(a) Simulation results of path planning for 5 USVs using **MCTP-ASOM** algorithm at low-layer.

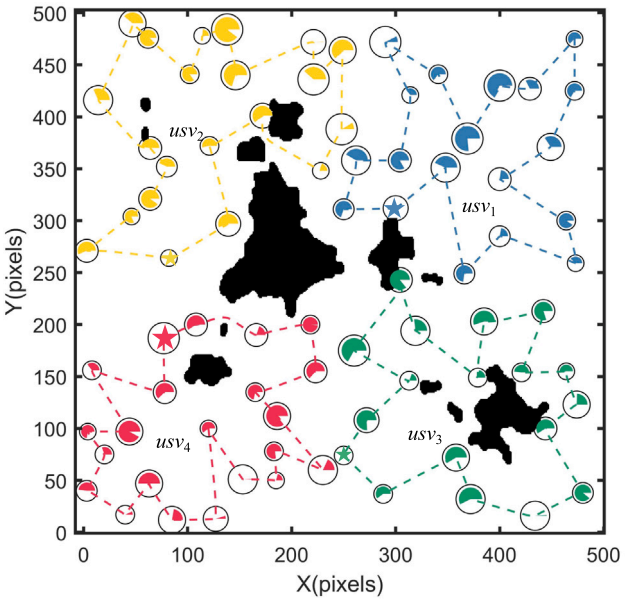


(b) Simulation results of bi-layer task planning for 5 USVs using SOM algorithm at low-layer.

Fig. 16. Simulation results of bi-layer task planning for 5 USVs executing 70 multi-constrained task points.



(a) Simulation results of path planning for 4 USVs using **MCTP-ASOM** algorithm at low-layer.



(b) Simulation results of path planning for 4 USVs using SOM algorithm at low-layer.

Fig. 17. Simulation results of bi-layer task planning for 4 USVs executing 80 multi-constrained task points.

References

Best, Graeme, Hollinger, Geoffrey A., 2020. Decentralised self-organising maps for multi-robot information gathering. In: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS, IEEE, pp. 4790–4797.

Jin, Xiaozhao, Er, Meng Joo, 2022. Cooperative path planning with priority target assignment and collision avoidance guidance for rescue unmanned surface vehicles in a complex ocean environment. Adv. Eng. Inform. 52, 101517.

Jin, Xiaozhao, Zhang, Gexiang, Yang, Ma, Peng, Xuechan, Shi, Binghua, 2022. Cooperative multi-task traversing with complex marine environment for multiple unmanned surface vehicles inspired by membrane computing. Ocean Eng. 266, 112586.

Kim, Heesu, Kim, Sang-Hyun, Jeon, Maro, Kim, JaeHak, Song, Soonseok, Paik, Kwang-Jun, 2017. A study on path optimization method of an unmanned surface vehicle under environmental loads using genetic algorithm. Ocean Eng. 142, 616–624.

Kohonen, Teuvo, 1982. Self-organized formation of topologically correct feature maps. Biol. Cybern. 43 (1), 59–69.

Kohonen, Teuvo, 1990. The self-organizing map. Proc. IEEE 78 (9), 1464–1480.

Liu, Yuanchang, Bucknall, Richard, 2018. Efficient multi-task allocation and path planning for unmanned surface vehicle in support of ocean operations. Neurocomputing 275, 1550–1566.

Liu, Yuanchang, Song, Rui, Bucknall, Richard, Zhang, Xinyu, 2019. Intelligent multi-task allocation and planning for multiple unmanned surface vehicles (USVs) using self-organising maps and fast marching method. Inform. Sci. 496, 180–197.

Liu, Zhixiang, Zhang, Youmin, Yu, Xiang, Yuan, Chi, 2016. Unmanned surface vehicles: An overview of developments and challenges. Annu. Rev. Control 41, 71–93.

Ma, Song, Guo, Weihong, Song, Rui, Liu, Yuanchang, 2021. Unsupervised learning based coordinated multi-task allocation for unmanned surface vehicles. Neurocomputing 420, 227–245.

Ma, Yong, Zhao, Yujiao, Qi, Xin, Zheng, Yuanzhou, Gan, Runze, 2018. Cooperative communication framework design for the unmanned aerial vehicles-unmanned surface vehicles formation. Adv. Mech. Eng. 10 (5), 1687814018773668.

MahmoudZadeh, Somaiyeh, Yazdani, Amirmehdi, 2022. A cooperative fault-tolerant mission planner system for unmanned surface vehicles in ocean sensor network monitoring and inspection. IEEE Trans. Veh. Technol. 72 (1), 1101–1115.

Mao, Shouqi, Yang, Ping, Gao, Diju, Bao, Chunteng, Wang, Zhenyang, 2023. A motion planning method for unmanned surface vehicle based on improved RRT algorithm. J. Mar. Sci. Eng. 11 (4), 687.

Miller, Clair E., Tucker, Albert W., Zemlin, Richard A., 1960. Integer programming formulation of traveling salesman problems. J. ACM 7 (4), 326–329.

Park, Jihoon, Kim, Sukkeun, Noh, Geemoon, Kim, Hyeongmin, Lee, Daewoo, Lee, Inwon, 2021. Mission planning and performance verification of an unmanned surface vehicle using a genetic algorithm. Int. J. Naval Archit. Ocean Eng. 13, 575–584.

Patten, Timothy, Zillich, Michael, Fitch, Robert, Vincze, Markus, Sukkarieh, Salah, 2015. Viewpoint evaluation for online 3-D active object classification. IEEE Robot. Autom. Lett. 1 (1), 73–81.

Sinaga, Kristina P., Yang, Miin-Shen, 2020. Unsupervised K-means clustering algorithm. IEEE Access 8, 80716–80727.

Singh, Yogang, Sharma, Sanjay, Sutton, Robert, Hatton, Daniel, Khan, Asiya, 2018. A constrained A* approach towards optimal path planning for an unmanned surface vehicle in a maritime environment containing dynamic obstacles and ocean currents. Ocean Eng. 169, 187–201.

Tan, Guoge, Zhuang, Jiayuan, Zou, Jin, Wan, Lei, 2022. Multi-type task allocation for multiple heterogeneous unmanned surface vehicles (USVs) based on the self-organizing map. Appl. Ocean Res. 126, 103262.

Wang, Ning, Xu, Hongwei, Li, Chengzhong, Yin, Jianchuan, 2021. Hierarchical path planning of unmanned surface vehicles: A fuzzy artificial potential field approach. Int. J. Fuzzy Syst. 23, 1797–1808.

Woo, Joohyun, Kim, Nakwan, 2020. Collision avoidance for an unmanned surface vehicle using deep reinforcement learning. Ocean Eng. 199, 107001.

Xia, Guoqing, Sun, Xianxin, Xia, Xiaoming, 2021. Multiple task assignment and path planning of a multiple unmanned surface vehicles system based on improved self-organizing mapping and improved genetic algorithm. J. Mar. Sci. Eng. 9 (6), 556.

Xue, Kai, Huang, Zhiqin, Wang, Ping, Xu, Zeyu, 2021. An exact algorithm for task allocation of multiple unmanned surface vehicles with minimum task time. J. Mar. Sci. Eng. 9 (8), 907.

Zereik, Enrica, Bibuli, Marco, Mišković, Nikola, Ridao, Pere, Pascoal, António, 2018. Challenges and future trends in marine robotics. Annu. Rev. Control 46, 350–368.

Zhu, Da-qi, Qu, Yun, Yang, Simon X., 2019. Multi-AUV SOM task allocation algorithm considering initial orientation and ocean current environment. Front. Inf. Technol. Electron. Eng. 20 (3), 330–341.

Zhu, Anmin, Yang, Simon X., 2010. An improved SOM-based approach to dynamic task assignment of multi-robots. In: 2010 8th World Congress on Intelligent Control and Automation. IEEE, pp. 2168–2173.